

Esame OS Internals (Cabodi)

Data: 17/01/2022

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Paginazione, Working Set e Codice C)

Contesto: Si consideri il seguente frammento di codice:

```
#define N 2048
float M[N][2*N], V[2*N*N], x;
int i, j, t;

for (t=0; t < 2*N*N; t++) {
    i = t / (2*N);
    j = t % (2*N);
    x = M[i][j];
    V[t] = x;
    M[i][j] = x * 2;
}
```

Il codice macchina generato da tali istruzioni viene eseguito su un sistema con gestione della memoria basata su demand paging (paginazione a richiesta), pagine da 2 KB, utilizzando una politica di sostituzione pagine working set (versione esatta) con $\delta = 10$. Si sa che:

- la dimensione di un `float` è di 32 bit;
- il segmento di codice (istruzioni in codice macchina) ha dimensioni inferiori a una pagina;
- `M` e `V` sono allocati ad indirizzi logici contigui (prima `M`, poi `V`), a partire dall'indirizzo logico `0xDB28A800`;
- la matrice `M` è allocata seguendo la strategia "row major", ovvero per righe (prima riga, seguita da seconda riga, ecc..).

Si risponda alle seguenti domande:

-) Quante pagine (e frame) sono necessarie per contenere la matrice e il vettore?
-) Supponiamo che le variabili `i`, `j`, `t` e `x` siano allocate in registri della CPU (accedendo a tali registri non si fa quindi alcun accesso in memoria RAM), quanti accessi totali a memoria $N_T = N_W + N_R$ (N_R per la lettura e N_W per la scrittura di dati) produce il programma proposto (non vanno conteggiati gli accessi a istruzioni)?
-) Sia N_T la quantità totale di riferimenti a dati in memoria (omettiamo per semplicità il fetch delle istruzioni) e sia N_L il numero di riferimenti (tra gli N_T totali) a una pagina già letta nei precedenti 10 accessi. Definiamo come località del programma per i dati il rapporto $L = N_L/N_T$. Si calcoli la località del programma proposto.
-) Si calcoli il numero di page fault generati dal programma proposto. (Motivare la risposta).

Domanda 2 (Disk Scheduling e SSTF)

Contesto: Si consideri un "disk scheduler" che gestisce richieste di accesso a blocchi logici su un disco di tipo HDD. La funzione `enqNextBlk` (abbreviata `eNB`) viene utilizzata per mettere nella coda la richiesta di accesso al blocco `blk`. La funzione `deqNextBlk` (`dNB`) viene chiamata per ottenere il nuovo blocco a cui fare accesso (il blocco schedulato) ogni volta che il disco è nuovamente pronto per una operazione. Per semplicità, non si distingue tra operazioni di lettura e scrittura.

Supponendo la seguente sequenza di chiamate, con coda di schedulazione inizialmente vuota e blocco corrente head (posizione iniziale della testina di lettura/scrittura) inizializzato al valore 500:

```
eNB (3050)
eNB (5150)
N = dNB()
eNB (1000)
eNB (1050)
N = dNB()
eNB (1200)
N = dNB()
N = dNB()
N = dNB()
```

Si risponda alle seguenti domande:

-) Si indichino i valori successivi di head e di N (accessi a blocchi su disco effettuati) e si rappresenti la coda dopo ogni chiamata a `enqNextBlk` (`eNB`). Si adotti una politica di schedulazione SSTF (Shortest Seek Time First), nella quale `dNB` ritorna il blocco più vicino (in una qualunque delle due direzioni) alla testina di lettura scrittura (head).
 -) Si calcoli la lunghezza del percorso totale della testina di lettura/scrittura, misurato in numero di cilindri (ogni cilindro contiene 50 blocchi).
-

Domanda 3 (Link dinamico, IPT e TLB)

Contesto: Si risponda alle seguenti domande sulla gestione della memoria:

-) Si consideri il caricamento dinamico (dynamic loading) e il link dinamico (dynamic linking). È possibile caricare dinamicamente un programma senza che sia necessario il dynamic linking? Il dynamic linking richiede che un programma sia anche caricabile dinamicamente (dynamic loading)?
-) Si spieghi brevemente perché un'Inverted Page Table necessiti di una tabella di HASH. Perché la soluzione di IPT + tabella di HASH è diversa da una soluzione con PT basata solamente su tabella di Hash?
-) Si consideri una CPU dotata di TLB: la TLB può contenere entry di più processi simultaneamente o è vincolata a contenere entry per un solo processo?

SECTION 2: OS161

Domanda 4 (System Call `waitpid`)

Contesto: Sia dato un sistema OS161. Si vuol realizzare il supporto (parziale, in quanto non si gestisce il parametro `flags`) per la system call `waitpid`, di cui si fornisce il prototipo:

```
pid_t waitpid (pid_t pid, int *returncode, int flags);
```

Si è già realizzata una funzione `proc_wait`, avente prototipo:

```
int proc_wait (struct proc *proc);
```

che dato il puntatore al descrittore di un processo, ne attende la fine (del processo) e ne ritorna lo stato di terminazione.

Quesito: Si realizzi una funzione `sys_waitpid`, che possa essere richiamata nella syscall mediante:

```
case SYS_waitpid:
    retval = sys_waitpid(tf->tf_a0, (userptr_t)tf->tf_a1);
    break;
```

ATTENZIONE: Di una eventuale tabella dei processi, è sufficiente fornire la dichiarazione e l'uso all'interno di `sys_waitpid`, nonché una breve descrizione a parole. Non è necessaria la gestione della tabella al boot e alla creazione/distruzione di un processo.

Domanda 5 (Strutture `uio` e `iovec`)

Contesto: Si consideri, in un sistema OS161, le definizioni delle `struct uio` e `iovec` qui riportate (`iovec` in forma semplificata ma equivalente all'originale):

```
struct uio {
    struct iovec *uio_iov;
    unsigned uio_iovcnt;
    off_t uio_offset;
    size_t uio_resid;
    enum uio_seg uio_segflg;
    enum uio_rw uio_rw;
    struct addrspace *uio_space;
};

struct iovec {
    void *iov_base;
    size_t iov_len;
};
```

Si risponda alle seguenti domande:

-) Spiegare brevemente la funzione svolta dai campi `uio_segflg`, `uio_rw` e `uio_space`.
-) Nel realizzare la system call `sys_write`, ci si aspetta che il campo `uio_segflg` indichi un segmento kernel o user? (Motivare).

-) Si dica poi perchè la `struct iovec`, puntata dal campo `uio_iov`, è una struct esterna. Sarebbe possibile evitare la struct aggiuntiva inserendo direttamente i campi `iov_base` e `iov_len` nella `struct uio`, come riportato qui sotto? (Motivare la risposta).

```
struct uio {  
    void *iov_base;  
    size_t iov_len;  
    unsigned uio_iovcnt;  
    off_t uio_offset;  
    size_t uio_resid;  
    enum uio_seg uio_segflg;  
    enum uio_rw uio_rw;  
    struct addrspace *uio_space;  
};
```

ANSWERS

Domanda 1 (Paginazione, Working Set e Codice C)

A) Pagine e frame necessari per M e V:

Un float occupa 4 Byte. Entrambe le strutture dati contengono $2 \times 2048 \times 2048 = 8.388.608$ elementi.

- Dimensione $M = 8.388.608 \times 4 \text{ B} = 33.554.432 \text{ B}$
- Dimensione $V = 8.388.608 \times 4 \text{ B} = 33.554.432 \text{ B}$

Dimensione totale = 67.108.864 B.

Poiché una pagina è da 2 KB (2048 B), il numero di pagine è:

$$\text{Pagine totali} = \frac{67.108.864}{2048} = \mathbf{32.768} \text{ pagine.}$$

B) Accessi totali in memoria (N_T):

Il ciclo iterativo viene eseguito $2 \times N \times N = 8.388.608$ volte.

All'interno del ciclo, escludendo le variabili in registro, si effettuano:

- $x = M[i][j]; \implies 1 \text{ Lettura } (N_R)$
- $V[t] = x; \implies 1 \text{ Scrittura } (N_W)$
- $M[i][j] = x * 2; \implies 1 \text{ Scrittura } (N_W)$

Accessi per iterazione = 3.

$$N_T = 3 \times 8.388.608 = \mathbf{25.165.824} \text{ accessi totali.}$$

C) Calcolo della località ($L = N_L/N_T$):

Gli array sono scansionati sequenzialmente. Ogni pagina (2048 B) contiene 512 float. Il programma cambia pagina (passando alla successiva di M e alla successiva di V) esattamente ogni 512 iterazioni.

- La finestra di località è $\Delta = 10$.
- Ad ogni iterazione si accede a M , poi V , poi di nuovo M .
- Quando si attraversa il confine di una pagina (ogni 512 iterazioni), la prima lettura della nuova pagina di M e la prima scrittura sulla nuova pagina di V causano un "Miss" di località, poiché tali pagine non figurano negli ultimi 10 accessi. La successiva scrittura su M è invece un "Hit", in quanto M è stata letta appena 2 accessi prima.
- Si verificano perciò esattamente **2 Miss ogni 512 iterazioni**.

Le iterazioni totali sono 8.388.608. I blocchi da 512 iterazioni sono $8.388.608/512 = 16.384$.

Numero totale di Miss = $2 \times 16.384 = \mathbf{32.768}$.

Il numero di Hit (N_L) è:

$$N_L = N_T - \text{Miss} = 25.165.824 - 32.768 = \mathbf{25.133.056}.$$

La località del programma è:

$$L = \frac{N_L}{N_T} = \frac{25.133.056}{25.165.824} = \frac{767}{768} \approx \mathbf{0.998698 \text{ (99,87\%)}}.$$

D) Numero di Page Fault:

Con l'algoritmo Working Set ($\Delta = 10$) e gli array scanditi sequenzialmente, una volta che una pagina viene abbandonata non verrà mai più riutilizzata. I page fault generati saranno esclusivamente quelli obbligatori (*compulsory misses*) al primo accesso di ogni singola pagina.

- Pagine utilizzate dall'array M = 16.384
- Pagine utilizzate dall'array V = 16.384

Essendo scandite entrambe parallelamente, ciascuna di queste genererà un page fault al suo primo caricamento.

$$\text{Totale Page Faults} = 16.384 + 16.384 = \mathbf{32.768 \text{ Page Faults}}.$$

Domanda 2 (Disk Scheduling e SSTF)

A) Sequenza delle operazioni e stato della coda:

Stato Iniziale: coda = {}, head = 500.

- eNB(3050) $\implies$ coda = {3050}, head = 500
- eNB(5150) $\implies$ coda = {3050, 5150}, head = 500
- N = dNB() $\implies$ Il più vicino a 500 è 3050.
N = 3050, coda = {5150}, head = 3050
- eNB(1000) $\implies$ coda = {5150, 1000}, head = 3050
- eNB(1050) $\implies$ coda = {5150, 1000, 1050}, head = 3050
- N = dNB() $\implies$ Distanze da 3050: (5150: 2100), (1000: 2050), (1050: 2000). Il più vicino è 1050.
N = 1050, coda = {5150, 1000}, head = 1050
- eNB(1200) $\implies$ coda = {5150, 1000, 1200}, head = 1050
- N = dNB() $\implies$ Distanze da 1050: (5150: 4100), (1000: 50), (1200: 150). Il più vicino è 1000.
N = 1000, coda = {5150, 1200}, head = 1000
- N = dNB() $\implies$ Distanze da 1000: (5150: 4150), (1200: 200). Il più vicino è 1200.
N = 1200, coda = {5150}, head = 1200
- N = dNB() $\implies$ L'unico rimasto è 5150.
N = 5150, coda = {}, head = 5150

I valori successivi assunti da N sono nell'ordine: **3050, 1050, 1000, 1200, 5150**.

B) Lunghezza del percorso totale della testina:

Il percorso effettuato in numero di blocchi è la somma dei valori assoluti degli spostamenti:

$$\begin{aligned}\text{Spostamenti (blocchi)} &= |3050 - 500| + |1050 - 3050| + |1000 - 1050| + |1200 - 1000| + |5150 - 1200| \\ &= 2550 + 2000 + 50 + 200 + 3950 \\ &= 8750 \text{ blocchi}\end{aligned}$$

Sapendo che ogni cilindro contiene 50 blocchi logici, la lunghezza del percorso totale espressa in cilindri è:

$$\text{Percorso Totale} = \frac{8750}{50} = \mathbf{175} \text{ cilindri.}$$

Domanda 3 (Link dinamico, IPT e TLB)

A) Dynamic Loading e Dynamic Linking:

Occorre distinguere il "Dynamic Loading" (le routine non vengono caricate in memoria finché non vengono effettivamente chiamate, gestito a livello utente) dal "Demand Paging" (caricamento pagina per pagina gestito dal SO a fronte di page fault).

- **Caricamento dinamico senza link dinamico? SÌ.**

È possibile sviluppare un programma interamente linkato staticamente (in cui tutto il codice delle librerie è già fuso nell'eseguibile su disco), ma strutturarlo internamente affinché carichi in RAM in modo dinamico i propri moduli solo quando vengono invocati.

- **Link dinamico richiede caricamento dinamico? SÌ.**

Il link dinamico consiste nel posticipare il collegamento delle librerie al tempo di esecuzione, inserendo nel codice un piccolo frammento detto *stub*. Quando lo *stub* viene eseguito, verifica se la libreria è in memoria; se non lo è, procede inevitabilmente a **caricarla dinamicamente**. Pertanto, il dynamic linking implica il dynamic loading della libreria di destinazione.

B) Inverted Page Table (IPT) e Hashing:

- **Perché necessita di Hash:**

L'IPT contiene una riga per ogni frame di memoria fisica esistente. Quando la CPU richiede la traduzione di una tupla (PID, Virtual Page Number), il sistema dovrebbe scandire l'intera IPT sequenzialmente, con un costo inaccettabile $O(N)$. L'uso di una funzione di Hash applicata alla tupla permette di accedere alla riga corretta in tempo $O(1)$.

- **Differenza tra IPT+Hash e Hash Page Table standard:**

La differenza fondamentale risiede nella struttura e nei limiti architetturali. In un'IPT, l'**indice stesso** della tabella corrisponde esattamente al numero di **frame fisico**. La dimensione dell'IPT è quindi strettamente limitata alla memoria fisica installata. In una *Hashed Page Table* classica, la tabella di hash non è legata alla RAM fisica, ma mappa i Virtual Page Numbers verso liste concatenate (per gestire le collisioni), all'interno delle quali è archiviato esplicitamente il mapping verso i frame fisici.

C) TLB e processi simultanei:

La TLB **può contenere** entry appartenenti a più processi simultaneamente, a condizione che l'architettura hardware (es. MIPS, ARM) supporti il campo **ASID (Address Space Identifier)** all'interno delle singole entry della TLB.

Motivazione: L'ASID funziona come una "targa" (simile al PID) che certifica a quale processo appartiene quella traduzione logico-fisica. Durante un accesso, l'hardware verifica il match sia sulla pagina logica sia sull'ASID del processo attualmente in esecuzione. Questo evita di dover invalidare (effettuare il *flush*) l'intera TLB ad ogni *Context Switch*, garantendo prestazioni notevolmente superiori. Solo le CPU più basilari (prive di ASID) sono vincolate a contenere entry per un solo processo alla volta.

Domanda 4 (System Call waitpid)

Dichiarazione della Tabella dei Processi:

Per supportare la conversione da PID a struttura processo, assumiamo l'esistenza di un array globale di puntatori nel kernel.

```
/* Tabella globale dei processi */
#define PID_MAX 1000
struct proc *processTable[PID_MAX];
```

Questa struttura agisce come una mappa $O(1)$: l'indice dell'array corrisponde direttamente al PID del processo. Le entry inutilizzate sono a NULL. (Nella realtà richiederebbe anche un lock per la sincronizzazione concorrente).

Implementazione di sys_waitpid:

La funzione riceve i parametri estratti dal *trapframe* (tf_a0 come PID e tf_a1 convertito in *userptr_t* per garantire la sicurezza della memoria).

```
#include <copyinout.h>
#include <kern/errno.h>

pid_t sys_waitpid(pid_t pid, userptr_t status_ptr) {
    struct proc *p;
    int exit_status;
    int err;

    /* 1. Controllo validità dei limiti del PID */
    if (pid < 0 || pid >= PID_MAX) {
        return -1; /* Errore: PID non valido */
    }

    /* 2. Recupero il processo dalla tabella */
    p = processTable[pid];
    if (p == NULL) {
        return -1; /* Errore: Processo inesistente (ESRCH) */
    }

    /* 3. Sospensione del chiamante in attesa della terminazione del figlio */
    exit_status = proc_wait(p);

    /* 4. Copia sicura dello stato verso lo User Space */
    /* Se status_ptr non è NULL, trasferiamo il dato usando copyout */
    if (status_ptr != NULL) {
        err = copyout(&exit_status, status_ptr, sizeof(int));
        if (err) {
            return -1; /* Errore: Puntatore utente invalido (EFAULT) */
        }
    }
}
```

```

    }
}

/* 5. In caso di successo, waitpid ritorna il PID del figlio atteso */
return pid;
}

```

Punti critici dell'implementazione:

- **Sicurezza Memoria (copyout):** È severamente vietato assegnare valori dereferenziando direttamente puntatori *User Space* (es. `*status_ptr = exit_status`). Si utilizza `copyout` affinché il kernel esegua la scrittura verificando i privilegi delle pagine di memoria.
- **Gestione Ritorno:** Il valore di ritorno della system call è il `pid` stesso, mentre il codice di terminazione viaggia attraverso il puntatore `status_ptr`.